System Design Interview Question

Background

You are a senior engineer at a multi-tenant data platform. Hundreds of enterprise customers continuously push records into your platform for processing and storage. You are designing the ingestion and processing layer that sits between what customers send and what gets stored.

The Setup

Customers send records in batches over HTTP. Each record belongs to a **dataset** — think of it like a table with a schema. Customers define that schema and can evolve it over time.

A record looks like this:

```json
{
  "customer_id": "acme-corp",
  "dataset_id": "orders",
  "schema_version": 2,
  "payload": {
    "order_id": "ord-8821",
    "amount": 4200,
    "currency": "USD",
    "placed_at": "2025-01-15T10:30:00Z"
  }
}
```

Schema fields are typed: `string`, `integer`, `float`, `boolean`, `timestamp`.

Scale: 50,000 records per second in aggregate across all customers. Individual customers can burst — one customer sending a large slow batch must not affect any other customer.

The Core Problems to Solve

Schema Management Customers register a schema for each dataset and can submit new versions as their data evolves. Your platform must decide whether to accept or reject a new version. Adding optional fields is allowed. Removing a field or changing a field's type is not.

Record Ingestion When a batch arrives, each record must be validated against the schema registered for that `dataset_id` at the stated `schema_version`. Valid records are written downstream. Invalid records are not.

Batch Report Every batch must produce a report: how many records succeeded, how many failed, and a clear reason for each failure.

What We Want to Discuss

- How you design and store the schema registry — and how schema lookups work without becoming a bottleneck at 50k rps
- How you process batches at this throughput — where the parallelism lives and how you think about the processing pipeline
- How you isolate customers from each other — what happens when one customer's batch is large, slow, or malformed
- What happens when things go wrong mid-batch — a schema lookup fails, a downstream write times out, half a batch is invalid
- How your design scales horizontally — what breaks first when you go to 10x

---

Take a few minutes to read through this. Start with the overall architecture, then we will go deeper into the areas you find most interesting.